MSO uniformisation over trees

April 10, 2025

1 Trees

Definition 1.1 (Tree)**.** A *tree* over an alphabet Σ is recursively defined as follows:

- a, where $a \in \Sigma$.
- $a(t, t')$, where $a \in \Sigma$ and t, t' are trees.

The set of all trees over Σ is Tree_Σ

Definition 1.2 (Nodes)**.** The *nodes* of a tree are defined as follows:

$$\mathrm{Nodes}\,(t) = \begin{cases} \varepsilon & \text{if } t = a \\ \varepsilon \cup \{0u \mid u \in \mathrm{Nodes}\,(t')\} \cup \{1u \mid u \in \mathrm{Nodes}\,(t'')\} & \text{if } t = a(t', t'') \end{cases}$$

Definition 1.3 (Leaves)**.** The *leaves* of a tree are defined as follows:

$$\mathrm{Leaves}\,(t) = \begin{cases} \varepsilon & \text{if } t = a \\ \{0u \mid u \in \mathrm{Leaves}\,(t')\} \cup \{1u \mid u \in \mathrm{Leaves}\,(t'')\} & \text{if } t = a(t', t'') \end{cases}$$

Remark 1.4. $\mathrm{Leaves}\,(t) \subseteq \mathrm{Nodes}\,(t)$

Definition 1.5. We note $t\,(n)$ the *label* of the node n in the tree t :

$$t\,(n) = \begin{cases} a & \text{if } t = a \text{ and } n = \varepsilon \\ a & \text{if } t = a(t', t'') \text{ and } n = \varepsilon \\ t'\,(m) & \text{if } t = a(t', t'') \text{ and } n = 0m \\ t''\,(m) & \text{if } t = a(t', t'') \text{ and } n = 1m \end{cases}$$

Definition 1.6. Let t be a tree and $x, y \in \mathrm{Nodes}\,(t)$, we define the *ancestor* relation as

$$x \sqsubseteq y \text{ if and only if } \exists z \in \mathrm{Nodes}\,(t)\,, y = xz$$

Definition 1.7. We note t^n the *subtree* corresponding to the node n on the tree t :

$$t^n = \begin{cases} t & \text{if } n = \varepsilon \\ t'^m & \text{if } t = a(t', t'') \text{ and } n = 0m \\ t''^m & \text{if } t = a(t', t'') \text{ and } n = 1m \end{cases}$$

2 Tree Automata

2.1 Non-Deterministic Bottom-Up Tree automata

Definition 2.1 (*Non-Deterministic Bottom-Up Tree automaton*)**.** A Non-Deterministic Bottom-Up Tree automaton (NBUA) is defined as a tuple $(\Sigma, Q, I, \Delta, F)$ where:

- Σ is an alphabet.
- Q is a finite set of *states*.
- $I \subseteq \Sigma \times Q$ correspond to the possible states of the leaves.
- $\Delta \subseteq Q \times Q \times \Sigma \times Q$ is the *transition relation*.
- $F \subseteq Q$ is the set of *final states*.

Definition 2.2 (Run of an automaton)**.** A *run* ρ of a NBUA A over a tree t is :

$$\begin{aligned} \rho : \text{Nodes}\,(t) &\rightarrow Q \\ (t\,(b)\,, \rho(n)) \in I &\quad \text{if} \quad n \in \text{Leaves}\,(t) \\ (\rho(n0), \rho(n1), t\,(n)\,, \rho(n)) \in \Delta &\quad \text{if} \quad n \in \text{Nodes}\,(t) \setminus \text{Leaves}\,(t) \end{aligned}$$

We say that ρ is *acceptant* if $\rho(\varepsilon) \in F$ and A accepts t if $F \cap \{\rho(\varepsilon) \mid \rho \text{ run of } A \text{ over } t\} \neq \emptyset$

Definition 2.3 (Language of an automaton)**.** Let A be a NBUA. Its *associated language* is defined as:

$$\mathcal{L}\,(A) = \{t \in \text{Tree}_\Sigma \mid \text{exists } \rho \text{ a accepting run of } A \text{ over } t\}$$

2.2 Deterministic Bottom-Up Tree automata

Definition 2.4 (*Deterministic Bottom-Up Tree automaton*)**.** A Deterministic Bottom-Up Tree automaton (DBUA) is NBUA that that verifies

$$\forall q, r, a, t, t' \ (q, r, a, t) \in \Delta \wedge (q, r, a, t') \in \Delta \implies t = t'$$

Remark 2.5. A DBUA can also be defined as as a tuple $(\Sigma, Q, \text{init}, \delta, F)$ where:

- Σ is an alphabet.
- Q is a finite set of states.
- init $: \Sigma \rightarrow Q$ is a function that initializes the states of the leaves.
- $\delta : Q \times Q \times \Sigma \rightarrow Q$ is the *transition map*.
- $F \subseteq Q$ is the set of final states.

TODO: init and δ should be the same We will show that the two definitions are equivalent.

Definition 2.6 (Interpretation of an automaton)**.** The *interpretation* of a DBUA A over an alphabet Σ is defined as follows:

$$\begin{array}{rcl} [\![A]\!] : \mathrm{Tree}_\Sigma & \to & Q \\ a & \mapsto & \mathrm{init}_a \\ a(t,t') & \mapsto & \delta_a([\![A]\!](t), [\![A]\!](t')) \end{array}$$

Remark 2.7. In the case of a DBUA there exists a unique run that corresponds to :

$$\begin{array}{rcl} \rho : \mathrm{Nodes}(t) & \to & Q \\ n & \mapsto & \mathrm{init}_{t(n)} \text{ if } n \in \mathrm{Leaves}(t) \\ n & \mapsto & \delta_{t(n)}(\rho(n0), \rho(n1)) \text{ if } n \in \mathrm{Nodes}(t) \setminus \mathrm{Leaves}(t) \end{array}$$

Lemma 2.8. *Let A be a DBUA,*

$$\mathcal{L}(A) = \{t \in \mathit{Tree}_\Sigma \mid [\![A]\!](t) \in F\}$$

Proof. We will prove by induction on a tree t that (a) there exists a run of A over t, and (b) for all runs ρ of A over t, $[\![A]\!](t) = \rho(\varepsilon)$. We proceed by case distinction.

- If $t = a$. Let ρ be defined by $\rho(\varepsilon) = \mathrm{init}_a$, then ρ is a run of A over t. We have proved (a). Consider now some run ρ of A over t. We have $\rho(\varepsilon) = \mathrm{init}_a = [\![A]\!](a) = [\![A]\!](t)$. We have proved (b).
- If $t = a(t', t'')$. By induction hypothesis (a), there exists runs ρ' and ρ'' on t' and t'' respectively. Let ρ be defined by $\rho(\varepsilon) = \delta_{t(\varepsilon)}(\rho'(\varepsilon), \rho''(\varepsilon))$, $\rho(0u) = \rho'(u)$ for all $u \in \mathrm{Nodes}(t')$ and $\rho(1u) = \rho''(u)$ for all $u \in \mathrm{Nodes}(t'')$. Then ρ is a run of A over t. We have proved (a).

  Consider now some run ρ of A over t. Let $\rho'(u) = \rho(0u)$ for all $u \in \mathrm{Nodes}(t')$ and $\rho''(u) = \rho(0u)$ for all $u \in \mathrm{Nodes}(t'')$. Then ρ' is a run of A over t' and ρ'' over t''. By induction hypothesis (b) twice, we know that $\rho'(\varepsilon) = [\![A]\!](t')$ and $\rho''(\varepsilon) = [\![A]\!](t'')$. We have now $\rho(\varepsilon) = \delta_{t(\varepsilon)}(\rho'(\varepsilon), \rho''(\varepsilon)) = \delta_{t(\varepsilon)}([\![A]\!](t'), [\![A]\!](t'')) = [\![A]\!](t)$. We have proved (b).

□

Definition 2.9. Let $A = (\Sigma, Q, I, \Delta, F)$ be a NBUA, we define the *power set automaton* of A, noted $\mathrm{Det}(A)$ as $\mathrm{Det}(A) = (\Sigma, \mathcal{P}(Q), \mathrm{init}, \delta, F')$ where:

- $\mathrm{init} : \Sigma \to \mathcal{P}(Q)$
  $a \mapsto \{q \in Q \mid (a, q) \in I\}$
- $\delta : \mathcal{P}(Q) \times \mathcal{P}(Q) \times \Sigma \to \mathcal{P}(Q)$
  $(X, Y, a) \mapsto \{q \in Q \mid (x, y, a, q) \in \Delta, x \in X, y \in Y\}$
- $F' = \{X \in \mathcal{P}(Q) \mid X \cap F \neq \emptyset\} \subseteq \mathcal{P}(Q)$

Theorem 2.10. *For all $N = (\Sigma, Q, I, \Delta, F)$ an NBUA, the power set automaton verifies that*

$$[\![\mathit{Det}(N)]\!](t) = \{\rho(\varepsilon) \mid \rho \text{ run of } N \text{ over } t\} \text{ and } \mathcal{L}(N) = \mathcal{L}(\mathit{Det}(N))$$

Proof. We will first show that for all tree t,

$$[\![\text{Det}\,(N)\,]\!]\,(t) = \{\rho(\varepsilon) \mid \rho \text{ run of } N \text{ over } t\} \ .$$

We will prove this property by induction over t. We proceed by case distinction.

- If $t = a$, then

$$\begin{aligned} \{\rho(\varepsilon) \mid \rho \text{ run of } N \text{ over } t\} &= \{q \in Q \mid (a, q) \in I\} \\ &= [\![\text{Det}\,(N)\,]\!]\,(t) \ . \end{aligned}$$

- If $t = a(t_1, t_2)$, then by the induction hypothesis we know that $[\![\text{Det}\,(N)\,]\!]\,(t_i) = q_i$ if and only if there exists a run over t_i ending at q_i, for $i \in \{1, 2\}$.

$$\begin{aligned} & q \in [\![\text{Det}\,(N)\,]\!]\,(t) \\ \text{if and only if} \quad & q \in \delta_a([\![\text{Det}\,(N)\,]\!]\,(t_1)\,, [\![\text{Det}\,(N)\,]\!]\,(t_2)) \\ \text{if and only if} \quad & q \in \{q \in Q \mid (x_1, x_2, a, q) \in \Delta, x_1 \in [\![\text{Det}\,(N)\,]\!]\,(t_1)\,, x_2 \in [\![\text{Det}\,(N)\,]\!]\,(t_2)\} \\ \text{if and only if} \quad & \exists x_1, x_2, (x_1, x_2, a, q) \in \Delta, x_1 \in [\![\text{Det}\,(N)\,]\!]\,(t_1)\,, x_2 \in [\![\text{Det}\,(N)\,]\!]\,(t_2) \\ \text{if and only if} \quad & \text{there exists a run over } t_i \text{ ending at } x_i \text{ for } i \in \{1, 2\} \text{ and } (x_1, x_2, a, q) \in \Delta \\ \text{if and only if} \quad & q \in \{\rho(\varepsilon) \mid \rho \text{ run of } N \text{ over } t\} \end{aligned}$$

Then $[\![\text{Det}\,(N)\,]\!]\,(t) = \{\rho(\varepsilon) \mid \rho \text{ run of } N \text{ over } t\}$.

We must now show that $\text{Det}\,(N)$ accepts t if and only if N accepts t.

$$\begin{aligned} N \text{ accepts } t \quad & \text{if and only if} \quad F \cap \{q \in Q \mid (a, q) \in I\} \neq \emptyset \\ & \text{if and only if} \quad F \cap [\![\text{Det}\,(N)\,]\!]\,(t) \neq \emptyset \\ & \text{if and only if} \quad [\![\text{Det}\,(N)\,]\!]\,(t) \in F' \\ & \text{if and only if} \quad \text{Det}\,(N) \text{ accepts } t \end{aligned}$$

and so $\mathcal{L}\,(N) = \mathcal{L}\,(\text{Det}\,(N))$.

TODO: Discuss proof □

2.3 Order-insensitive

Definition 2.11. An NBUA of transition relation Δ is *order-insensitive* if for all transitions $(p, q, a, r) \in \Delta$, $(q, p, a, r) \in \Delta$.

Remark 2.12. For a DBUA of transition map δ, it is order-insensitive if and only if $\delta_a(p, q) = \delta_a(q, p)$ for all states p, q and letter a.

Lemma 2.13. *If $N = (\Sigma, Q, I, \Delta, F)$ a NBUA is order-insensitive, then Det(N) is also order-insensitive.*

Proof.

$$\begin{aligned}
\delta_a(X,Y) &= \{q \in Q \mid (x,y,a,q) \in \Delta, x \in X, y \in Y\} \\
&= \{q \in Q \mid (y,x,a,q) \in \Delta, x \in X, y \in Y\} \quad (N \textit{ is order-insensitive}) \\
&= \delta_a(Y,X)
\end{aligned}$$

□

Definition 2.14. Let $A = (\Sigma, Q, \mathrm{init}, \delta, F)$ a DBUA, let $\mathrm{S}_A \subseteq Q$ be the smallest set such that

1. $\mathrm{init}(a) \in \mathrm{S}_A$ for all $a \in \Sigma$,
2. $\delta(p,q,a) \in \mathrm{S}_A$ for all $p, q \in \mathrm{S}_A$ and $a \in \Sigma$.

Lemma 2.15. *Let A a DBUA, then*

$$S_A = \{[\![A]\!](t) \mid t \in \mathit{Tree}_\Sigma\}$$

Proof.

- $\boxed{\subseteq}$
  We will prove by induction that $q \in \mathrm{S}_A \implies$ there exists $t \in \mathrm{Tree}_\Sigma$ such that $[\![A]\!](t) = q$.
  - If $q = \mathrm{init}(a) \in \mathrm{S}_A$, then the tree a works : $[\![A]\!](a) = \mathrm{init}(a) = q$.
  - If $q = \delta(p,q,a) \in \mathrm{S}_A$ where $p, q \in \mathrm{S}_A$ and $a \in \Sigma$. Then by the induction hypothesis there exists t' such that $[\![A]\!](t') = p$ and t'' such that $[\![A]\!](t'') = q$. Then we have that $t = a(t', t'')$ verifies $[\![A]\!](t) = q$.
- $\boxed{\supseteq}$
  We will prove it by induction on the structure of a tree t. We proceed by case distinction.
  - If $t = a$, then we have $[\![A]\!](a) = \mathrm{init}(a) \in \mathrm{S}_A$
  - If $t = a(t', t'')$, we have $[\![A]\!](t) = \delta([\![A]\!](t'), [\![A]\!](t''), a) \in \mathrm{S}_A$ because by the induction hypothesis $[\![A]\!](t') \in \mathrm{S}_A$ and $[\![A]\!](t'') \in \mathrm{S}_A$.

□

2.4 Universality

Definition 2.16. Let A a NBUA we say that it is *universal* if it accepts every tree.

Theorem 2.17. *$A = (\Sigma, Q, \mathit{init}, \delta, F)$ is universal if and only if $S_A \subseteq F$.*

Proof.

- $\boxed{\Rightarrow}$
  We suppose A universal, then for all $t \in \mathrm{Tree}_\Sigma$ we have that $[\![A]\!](t) \in F$, because A accepts t. Since $\mathrm{S}_A = \{[\![A]\!](t) \mid t \in \mathrm{Tree}_\Sigma\}$ we have that $\mathrm{S}_A \subseteq F$.

- $\boxed{\Leftarrow}$
  We suppose $\mathrm{S}_A \subseteq F$. Let t be a tree, then $[\![A]\!](t) \in \mathrm{S}_A \subseteq F$, so $[\![A]\!](t) \in F$ and A accepts t. We have proven that A is universal.

$\square$

Corollary 2.18. *A saturation algorithm based on S_A decides universality.*

3 MSO

Definition 3.1. Let $t \in \mathrm{Tree}_\Sigma$, we define a model of t as :

$$\mathcal{M}(t) := (\mathrm{Nodes}(t), \sqsubseteq, (a(x))_{a\in\Sigma})$$

where : $a(x)$ if and only if $t(x) = a$

We will discuss some examples of MSO formulas.

Example 3.2. The formula $\mathrm{leaf}(x) = \forall y, y \sqsubseteq x$ encodes the fact that x is a leaf.

Example 3.3. x is the root of the tree is equivalent to :

$$\mathrm{root}(x) = \forall y, x \sqsubseteq y$$

Example 3.4. The following formula encodes the fact that x and y are the immediate children of z:

$$\mathrm{children}(x, y, z) = x \neq y \wedge z \sqsubseteq x \wedge z \sqsubseteq y \wedge \forall w\, ((z \sqsubseteq w \wedge (w \sqsubseteq x \vee w \sqsubseteq y)) \to (w = x \vee w = y))$$

Example 3.5. The formula $\mathrm{equal}(X, T) = \forall x, (X(x) \to Y(x)) \wedge (Y(x) \to X(x))$ encodes the fact that X and Y are the same set.

3.1 Automaton → MSO

Lemma 3.6. *If A is a order-insensitive NBUA then there exists a MSO formula φ such that for all $t \in \mathrm{Tree}_\Sigma$,*

$$\mathcal{M}(t) \models \varphi \text{ if and only if } t \in \mathcal{L}(A)$$

Proof. Let $A = (\Sigma, Q, I, \Delta, F)$ be a NBUA, and $k = |Q|$.

$$\varphi = \exists X_0 \dots \exists X_{k-1} \qquad \forall x \left(\bigvee_{i=0}^{k-1} X_i(x) \right) \tag{3.1}$$

$$\wedge \quad \forall x \left(\bigwedge_{0 \le i < j \le k-1} \neg(X_i(x) \wedge X_j(x)) \right) \tag{3.2}$$

$$\wedge \quad \forall x \left(\text{leaf}(x) \to \bigvee_{(a,i) \in I} X_i(x) \right) \tag{3.3}$$

$$\wedge \quad \forall x \forall y \forall z \left(\text{children}(x, y, z) \to \bigvee_{(p,q,a,r) \in \Delta} (X_p(x) \wedge X_q(y) \wedge a(z) \wedge X_r(z)) \right) \tag{3.4}$$

$$\wedge \quad \forall x \left(\text{root}(x) \to \bigvee_{q \in F} X_q(x) \right) \tag{3.5}$$

TODO: fix overfull
$j \in X_i$ if and only if , for a given run, the state i is reached from the node j.
The formula φ encodes the following properties:

1. Every node has an associated state.
2. A node cannot be in two states at the same time.
3. The transitions originate from the leaves.
4. The transitions of inner nodes ensure that for every trio of nodes, where two are children of the third, there exists a transition from the children to the parent.
5. Finally, the word is accepted if and only if the state associated with the root node is a final state.

From this we deduce that

$$\mathcal{M}(t) \models \varphi \text{ if and only if } t \in \mathcal{L}(A)$$

□

3.2 MSO → automaton

Definition 3.7. The *cartesian product automata* of two NBUA $A_1 = (\Sigma, Q_1, I_1, \Delta_1, F_1)$ and $A_1 = (\Sigma, Q_2, I_2, \Delta_2, F_2)$ with final states $F' \subseteq Q_1 \times Q_2$ is :

$$A_1 \times A_2 = (\Sigma, Q_1 \times Q_2, I_1 \times I_2, \Delta_3, F')$$

where $\Delta_3 = \{((q_1, q_2), (p_1, p_2), (a_1, a_2), (r_1, r_2) \mid (q_1, p_1, a_1, r_1) \in \Delta_1, (q_2, p_2, a_2, r_2) \in \Delta_2\}$

Lemma 3.8. *Let* $A_1 = (\Sigma, Q_1, I_1, \Delta_1, F_1)$ *and* $A_1 = (\Sigma, Q_2, I_2, \Delta_2, F_2)$ *be two NBUA, then* $A_3 = A_1 \times A_2$ *with final states* $F_1 \times F_2$ *verifies that* $\mathcal{L}(A_3) = \mathcal{L}(A_1) \cap \mathcal{L}(A_2)$

Lemma 3.9. *Let* $A_1 = (\Sigma, Q_1, I_1, \Delta_1, F_1)$ *and* $A_1 = (\Sigma, Q_2, I_2, \Delta_2, F_2)$ *be two NBUA, then*

$$A_3 = (\Sigma, Q_1 \uplus Q_2, I_1 \uplus I_2, \Delta_1 \uplus \Delta_2, F_1 \uplus F_2)$$

verifies that $\mathcal{L}(A_3) = \mathcal{L}(A_1) \cup \mathcal{L}(A_2)$

Lemma 3.10. *Let* $A = (\Sigma, Q, I, \Delta, F)$ *the NBUA* $A' = (\Sigma, Q, I, \Delta, Q \cap F)$ *verifies that* $\mathcal{L}(A') = (\mathcal{L}(A))^{\complement}$

Lemma 3.11. *Over the alphabet* $\Sigma \times \{0,1\}$ *the automaton* A *described bellow can decide the formula* $a(x)$*, where* $a \in \Sigma$ *and* x *is a free variable.*

$$A = (\Sigma \times \{0,1\}, \{Yes,\ No\}, init, \delta, Yes)$$

The following function definitions should be interpreted as a pattern match, with priority for the first rule :

- $$\begin{aligned} init : \Sigma \times \{0,1\} &\to \mathcal{P}(Q) \\ (a,1) &\mapsto Yes \\ _ &\mapsto No \end{aligned}$$
- $$\begin{aligned} \delta : Q \times Q \times (\Sigma \times \{0,1\}) &\to Q \\ (_, _, (a,1)) &\mapsto Yes \\ (No, No, _) &\mapsto No \\ _ &\mapsto Yes \end{aligned}$$

Lemma 3.12. *Over the alphabet* $\Sigma \times \{0,1\}^2$ *the automaton* A *described bellow can decide the formula* $x \sqsubseteq y$*, where* x *and* y *are free variables.*

$$A = (\Sigma \times \{0,1\}^2, \{Yes,\ No,\ FoundY\}, init, \delta, Yes)$$

The following function definitions should be interpreted as a pattern match, with priority for the first rule :

- $$\begin{aligned} init : \Sigma \times \{0,1\}^2 &\to \mathcal{P}(Q) \\ (_, _, 1) &\mapsto FoundY \\ _ &\mapsto No \end{aligned}$$

- $\delta : Q \times Q \times (\Sigma \times \{0,1\}^2) \to Q$

$$
\begin{aligned}
(\mathit{FoundY}, _, (_, 1, 0)) &\mapsto \mathit{Yes} \\
(_, \mathit{FoundY}, (_, 1, 0)) &\mapsto \mathit{Yes} \\
(\mathit{FoundY}, _, (_, 0, 0)) &\mapsto \mathit{FoundY} \\
(_, \mathit{FoundY}, (_, 0, 0)) &\mapsto \mathit{FoundY} \\
(\mathit{Yes}, _, _) &\mapsto \mathit{Yes} \\
(_, \mathit{Yes}, _) &\mapsto \mathit{Yes} \\
(_, _, (_, 0, 1)) &\mapsto \mathit{FoundY} \\
_ &\mapsto \mathit{No}
\end{aligned}
$$

Lemma 3.13. *If φ is a MSO formula there exists A a NBUA such that for all $t \in \mathit{Tree}_\Sigma$,*

$$\mathcal{M}(t) \models \varphi \text{ if and only if } t \in \mathcal{L}(A)$$

Proof. The proof procedes by induction on the structure of φ.

If φ is an atomic formula, then the lemmas above decide it.

If φ is of the form $\psi_1 \wedge \psi_2$ then we can compute the intersection between the automata for ψ_1 and ψ_2, obtained using the induction hypothesis. We can deal with the disjuction using the union instead of the intersection.

If φ is of the form $\neg\psi$ then we can compute the complement of the automaton corresponding to ψ.

We have to deal with the case $\varphi = \exists x.\psi(x)$. We can find a NBUA $A = (\Sigma \times \{0,1\}^n, Q, I, \Delta, F)$ for ψ where x is treated as a free variable.

Then the automaton $A' = (\Sigma \times \{0,1\}^{n-1}, Q \times \{0,1\}, I \times \{0\}, \Delta', F \times \{1\})$ where

$$
\begin{aligned}
\Delta' = &\left\{ \begin{array}{l} ((q,u),(q',u'),(a,X),(q'',u'')) \mid u, u' \in \{0,1\}, \\ \quad X' \text{ is } X \text{ where we add 0 at position } i, \\ \quad u'' = u \wedge u', \quad (q, q', (a, X'), q'') \in \Delta \end{array} \right\} \\
\cup &\left\{ \begin{array}{l} ((q,0),(q',0),(a,X),(q'',1)) \mid \\ \quad X' \text{ is } X \text{ where we add 1 at position } i, \\ \quad (q, q', (a, X'), q'') \in \Delta \end{array} \right\}.
\end{aligned}
$$

and i corresponds to the variable x, can decide φ. If existential quantification is applied to a second-order variable, the automaton remains unchanged. However, instead of considering a single variable in the transition function, look at all the variables within the given set.

□

4 Uniformisation

Definition 4.1 (Uniformiser)**.** We say that a formula $\psi(X)$ is a *uniformiser* of $\varphi(X)$ if for all trees t, the following conditions hold:

1. There exists at most one set X such that $\mathcal{M}(t) \models \psi(X)$.

2. If there exists a set X such that $\mathcal{M}(t) \models \varphi(X)$, then there exists a set Y such that $\mathcal{M}(t) \models \psi(Y) \wedge \varphi(Y)$.

Lemma 4.2. *$\psi(X)$ is a uniformiser of $\varphi(X)$ if and only if for all trees t, the following holds:*

$$\begin{aligned} \mathcal{M}(t) \models \quad & \forall X \forall Y \Big((\psi(X) \wedge \psi(Y)) \to X = Y \Big) \\ \wedge \quad & (\exists X\, \varphi(X)) \to (\exists X\, \psi(X)) \\ \wedge \quad & (\forall X\, \psi(X) \to \varphi(X)). \end{aligned}$$

Proof. We analyze the conditions in the first formulation and show their correspondence to the logical constraints in the second formulation:

1. The formula $\forall X \forall Y\, (\psi(X) \wedge \psi(Y) \to X = Y)$ ensures that ψ has at most one solution in $\mathcal{M}(t)$. This directly corresponds to the first condition in the original problem statement.

2. The formula $\exists X\, \varphi(X) \to \exists X\, \psi(X)$ expresses that whenever φ has a solution, ψ must also have a solution. However, this does not require ψ to hold for the same one as φ.

   The formula $\forall X\, (\psi(X) \to \varphi(X))$ ensures that whenever ψ holds for a set, φ must also hold for that same set.

   Combining the last two constraints, we obtain the second condition: whenever there exists a set satisfying φ, there must exist a set satisfying both ψ and φ.

□

Problem 1. *Can we find a uniformiser for every MSO formula $\varphi(X)$?*

In an intuitive way, the above problem is equivalent to asking if the following formula (it is not a real MSO formula because we are quantifying over a formula) holds for every φ :

$$(\exists X.\, \varphi(X)) \to (\exists \psi(_).\, \varphi(\{x \mid \psi(x)\})$$

We will start by studying how we can decide for two formulas $\varphi(X)$ and $\psi(X)$ if the formula $\Psi = \exists X.\varphi(X) \to \exists Y.\varphi'(Y)$ is satisfiable for all trees. We will build a DBUA than can decide if a tree satisfies Ψ. Thanks to the construction from Lemma 3.13, we can build a DBUA $A = (\Sigma \times \{0,1\}, Q, \mathrm{init}, \delta, F)$ for $\varphi(X)$ and another one $A' = (\Sigma' \times \{0,1\}, Q', \mathrm{init}', \delta', F')$ for $\psi(Y)$.

We can now build a new DBUA $A'' = (\Sigma, \mathcal{P}(Q) \times \mathcal{P}(Q'), \mathrm{init}'', \delta'', F'')$ where

- $\mathrm{init}'' : \Sigma \to \mathcal{P}(Q) \times \mathcal{P}(Q')$
  $a \mapsto (\{\mathrm{init}(a,b) \mid b \in \{0,1\}\}, \{\mathrm{init}'(a,b) \mid b \in \{0,1\}\})$

- $\delta'' : (\mathcal{P}(Q) \times \mathcal{P}(Q'))^2 \times \Sigma \to \mathcal{P}(Q) \times \mathcal{P}(Q')$
  $$((X,X'),(Y,Y'),a) \mapsto \begin{array}{c} (\{\delta(x,y,(a,b))\} \mid \{x \in X, y \in Y, b \in \{0,1\}\}, \\ \{\delta'(x',y',(a,b))\} \mid \{x' \in X', y' \in Y', b \in \{0,1\}\}) \end{array}$$

- $F'' = \{(X,X') \mid (X \cap F \neq \emptyset) \to (X' \cap F' \neq \emptyset)\}$

This automaton accepts a tree if and only if that tree is a model for Ψ. In particular, using the decision procedure from Corollary 2.18 we can decide if Ψ is satisfied by all models.

We are interested in finding an automaton that, if it is universal, then φ is uniformisable. The approach we use consists on slightly modifying the automaton for $\exists X.\varphi(X) \to \exists Y.\varphi(Y)$ in a way that it guarantees that Y is definable. We will use the same construction as for Ψ but modifying the transition function.

Definition 4.3. Let $\varphi(X)$ a MSO formula, and $A = (\Sigma \times \{0,1\}, Q, \mathrm{init}, \delta, F)$ the DBUA for $\varphi(X)$, we call $U = (\Sigma, \mathcal{P}(Q) \times \mathcal{P}(Q), \mathrm{init}', \delta', F')$ the unifier automaton of φ, where TODO: φ should be a NBUA and not a DBUA

- $\mathrm{init}' : \Sigma \to \mathcal{P}(Q) \times \mathcal{P}(Q)$
  $$a \mapsto (\{\mathrm{init}(a,b) \mid b \in \{0,1\}\}, \{\mathrm{init}(a,b) \mid b \in \{0,1\}\})$$
- $\delta_{U_2} : \mathcal{P}(Q)^2 \times \mathcal{P}(Q)^2 \times \Sigma \to \mathcal{P}(Q)^2$ TODO:
  $$((X,X'),(Y,Y'),a) \mapsto (\{\delta(X,Y,(a,b))\} \mid \{x \in X, y \in Y, b \in \{0,1\}\}, \delta_U(X',Y',a))$$
  introduce notation $\delta(X,Y,(a,*))$ TODO: rename to δ_{U_2}
- $\delta_U : \mathcal{P}(Q) \times \mathcal{P}(Q) \times \Sigma \to \mathcal{P}(Q)$
  $$(X',Y',a) \mapsto \begin{cases} \{\delta(p,q,(a,b))\} \mid \{p \in X', q \in Y', a \in \Sigma, b \in \{0,1\}, p = q\} & \text{if } X' = Y' \\ \{\delta(p,q,(a,b))\} \mid \{p \in X', q \in Y', a \in \Sigma, b \in \{0,1\}\} & \text{otherwise} \end{cases}$$
- $F' = \{(X,X') \mid (X \cap F \neq \emptyset) \to (X' \cap F \neq \emptyset)\}$

Lemma 4.4. *Let $\varphi(X)$ be and MSO formula, if the unifier automaton U of φ is universal then φ is uniformisable.*

Proof. We need to exhibit a ψ that is a uniformiser for $\varphi(X)$. We consider $U = (\Sigma, \mathcal{P}(Q) \times \mathcal{P}(Q), \mathrm{init}', \delta', F')$ the unifier automaton of φ, and $A = (\Sigma \times \{0,1\}, Q, \mathrm{init}, \delta, F)$ the automaton for $\varphi(X)$.

Given a set of states P, from Lemma 3.13 there exists a formula $\mathrm{u_state}_P(x)$ such that TODO: fix ref

$$\mathcal{M}(t) \models \mathrm{u_state}_(x) \text{ if and only if } \delta_U(t^x) = P.$$

TODO: Explain somewhere the notation $\delta(t)$

We first introduce a choice function $\mathrm{choice} : \mathcal{P}(Q) \times \mathcal{P}(Q) \times Q \times \Sigma \to Q \times Q \times \{0,1\}$ that given (X',Y',r,a) chooses a triple (b,p,q) such that, if $r \in \delta_{U_2}(X',Y',a)$ then $\delta(p,q,(a,b)) = r$ where $p \in X', q \in Y'$ and if $X' = Y'$ then $p = q$. We overload its type to also be $\mathrm{choice} : Q \times \Sigma \to \{0,1\}$, in which case it chooses b such that $p = \mathrm{init}((a,b))$. The last overload is used to choose from a final state: $\mathrm{choice} : \mathcal{P}(Q) \to Q$ where $\mathrm{choice}(X) = q$ such that if $X \cap F \neq \emptyset$ then $q \in X \cap F$. If the premises are not satisfied in any of the cases, then choice returns an arbitrary value. TODO: 3 different functions instead of one overloaded

We define the set X_U ant the run ρ_U over the tree (t,X) by induction on the depth of the node as follows: TODO: introduce notation for annotated trees

- For the root, we define $\rho_U(\varepsilon) = \mathrm{choice}(\delta_{U_2}(t))$.
- For all inner nodes x with children y and z. Let (p,q,v) be $\mathrm{choice}(\delta_{U_2}(t^y), \delta_{U_2}(t^z), \rho_U(x), t(x))$, we define $\rho_U(y) = p$, $\rho_U(z) = q$, and $x \in X_U$ if and only if $b = 1$.

- For all leaves x, let b be $b = \text{choice}(p, t(x))$ we define $\rho_U(x) = p$, where $x \in X_U$ if and only if $b = 1$.

We claim now that the ρ_U is well defined, unique, and indeed a run of A over (t, X_U). The proof procedes by top-down induction.

- For ε, the root of t, we have that $\rho_U(\varepsilon) = \text{choice}(\delta_{U_2}(t))$, since choice is deterministic, $\rho_U(\varepsilon)$ is unique.
- For the inner nodes x with children y and z, there exists $r \in Q$, such that $\rho_U(x) = r$ by the induction hypothesis. Then, since choice is deterministic, there exists a unique tuple (p, q, b) such that $(p, q, b) = \text{choice}(\delta_{U_2}(t^y), \delta_{U_2}(t^z), r, t(x))$. We have that $\rho_U(y) = p$ and $\rho_U(z) = q$, which verify that $\delta(p, q, (t(x), b)) = r$. We also have that $x \in X_U$ if and only if $b = 1$. This is then a valid run of A over (t, X_U)
- For the leaves x of label a, by the induction hypothesis we have that $b = \text{choice}(a, \rho_U(x))$ which is well defined since $a \in \Sigma$ and unique. We also have that $x \in X$ if and only if $b = 1$ and $\rho(x) = \text{init}((a, b))$.

The claim is proved.

We are now ready to define a formula $\text{unif}(X)$ that witnesses that Φ is uniformisable. This formula simply states using the MSO-syntax the existence of the run ρ_U described above, *i.e.* ,

$$\mathcal{M}(t) \models \text{unif}(X) \text{ if and only if } X = X_U$$

Let $k = |Q|$. We define 3 subformulas :

$$\begin{aligned}
\text{single_state}(X_{p_1}, \ldots, X_{p_k}) &= \forall x, \bigvee_{p_i \in \text{Nodes}(t)} X_{p_i}(x) \\
&\wedge \forall x, \bigwedge_{p_i, p_j \in \text{Nodes}(t), i \neq j} \neg(X_{p_i}(x) \wedge X_{p_j}(x))
\end{aligned}$$

$$\text{root_node}(X_{p_1}, \ldots, X_{p_k}) = \forall x, \text{root}(x) \rightarrow (\bigvee_{\left(\substack{p_i \in \\ \delta_U(\varepsilon)}\right)} X_{p_i}(x))$$

$$\begin{aligned}
\text{inner_nodes}(X_{p_1}, \ldots, X_{p_k}, X) &= \forall x \forall y, \forall z, \text{children}(x, y, z) \rightarrow \\
&\bigvee_{\substack{(P', Q', a, R') \in \delta_U \\ r \in R' \\ \text{choice}(P', Q', a, r) = (p, q, b)}} \Big(\text{u_state}_{P'}(x) \wedge \text{u_state}_{Q'}(y) \\
&\wedge X_p(x) \wedge X_q(y) \wedge a(z) \wedge X_r(z) \\
&\wedge (\bigwedge_{b=1} X(x)) \wedge (\bigwedge_{b=0} \neg X(x))\Big)
\end{aligned}$$

TODO: Leaves, but I'll do it when init becomes δ.

We can now define unif(X):

$$\begin{aligned}\text{unif}(X) &= \exists X_{p_1}, \ldots, \exists X_{p_k}, \text{single_state}(X_{p_1}, \ldots, X_{p_k}) \\ &\wedge \text{root_node}(X_{p_1}, \ldots, X_{p_k}) \\ &\wedge \text{inner_nodes}(X_{p_1}, \ldots, X_{p_k}, X)\end{aligned}$$

TODO: Prove that unif satisfies the main property

The last thing we have to prove is that there is a unique ρ_U, X that satisfy the formula. TODO □

Lemma 4.5. *Let $\varphi(X)$ be and MSO formula, if φ is uniformisable then the unifier automaton U of φ is universal.*

Proof. We will prove the contraposition. TODO □